Who am I?

- Computer Engineer by education
- Started in software development in High School using VB6
- Worked in PCB design, board bring up, embedded firmware, RTOS, build systems, systems integration, cloud services, and more.

What I'll cover

Getting started in reverse engineering monolithic firmware with Ghidra

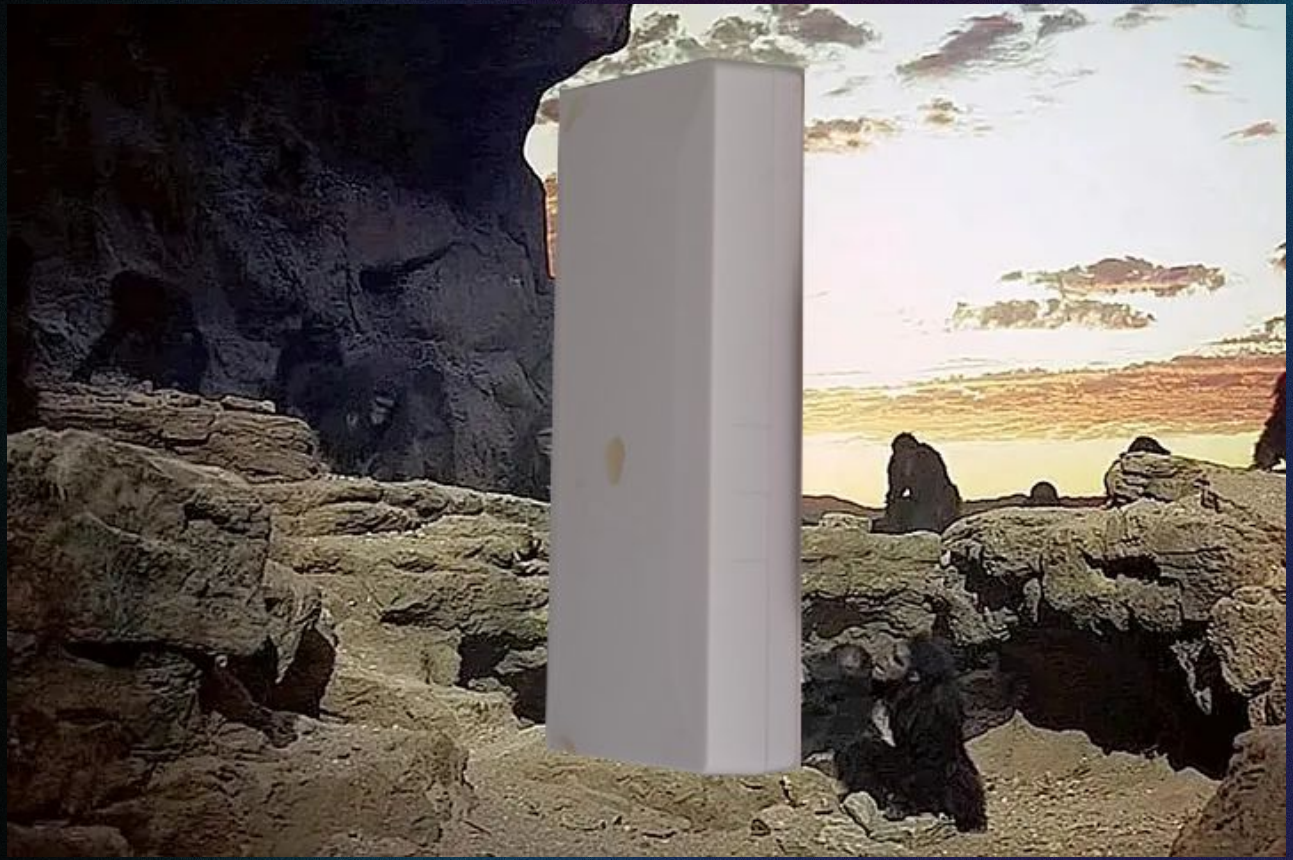


THE PROBLEM: KIDS + CATS + DOORS AND WINDOWS





SOLUTION



Before the IoT, there were 345MHz wireless sensors

- 1 way communication, TX, no RX
- Ran on batteries for years (CR2, CR123, CR2032's, etc)
- 345MHz penetrates walls and is relatively clean spectrum
- No updating firmware
- Relatively cheap, $20 in 2014
- Relatively simple design and firmware
- The either work or they don't – no debugging network routes or app based enrollment
- Millions of these already in homes across the world, may of which are gathering dust
- Low data rate, low update frequency

Let’s see if I can get these working in Home Assistant

1. Buy SDR
2. Configure MQTT broker in Home Assistant
3. Run rtl_433 (open source software to decode 345MHz signals
4. Figure out the MQTT topic that makes it all work
5. ?
6. Profit

Uh-oh, some of these are different





In 2014, Vivint released their “encrypted” sensors

Without a method to validate the source of incoming packets, spoofing 345MHz not only became economical, it became trivial.

Even key fobs that can arm or disarm a system broadcast their commands in the clear

Solution? Encrypt the important bits

Bonus: New devices only works with Vivint systems, prevents system takeovers

Also: Prevents devices from working with Home Assistant

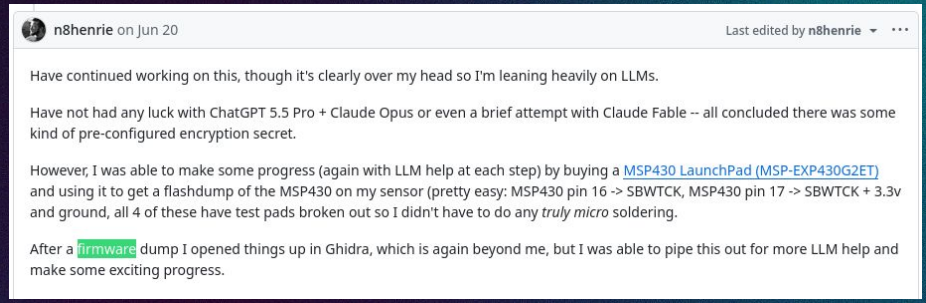
Don the hacker hoodie

Hey look! Somebody else successfully extracted the firmware. Thank you n8thenrie!

Turns out my msp430 dev kit from 2012 works as a debugger

Plenty of other tutorials of how to get your hands on firmware.

PS: It won’t always be this easy.



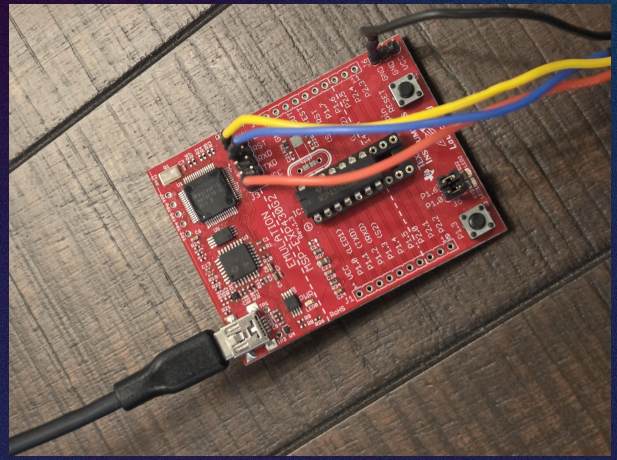
n8henrie on Jun 20 — Last edited by n8henrie

Have continued working on this, though it's clearly over my head so I'm leaning heavily on LLMs.

Have not had any luck with ChatGPT 5.5 Pro + Claude Opus or even a brief attempt with Claude Fable -- all concluded there was some kind of pre-configured encryption secret.

However, I was able to make some progress (again with LLM help at each step) by buying a MSP430 LaunchPad (MSP-EXP430G2ET) and using it to get a flashdump of the MSP430 on my sensor (pretty easy: MSP430 pin 16 -> SBWTCK, MSP430 pin 17 -> SBWTCK + 3.3v and ground, all 4 of these have test pads broken out so I didn't have to do any *truly micro* soldering.

After a firmware dump I opened things up in Ghidra, which is again beyond me, but I was able to pipe this out for more LLM help and make some exciting progress.

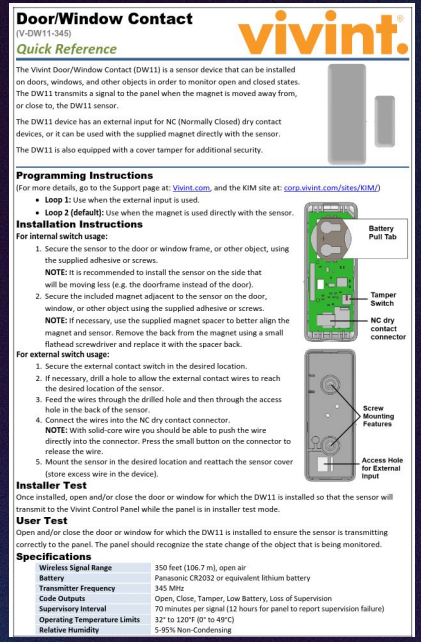


Here's what I started with

- Same 345MHz frequency and encoding - OOK with Manchester encoding
- Instead of Honeywell 64 bit packet, now uses 96 bit packet
- No code protection on the device, able to extract firmware
- Packet format already mostly decoded:

All I had to do was figure out six bits in the payload and nibble that didn't seem to be part of the CRC.

Easy enough?



Door/Window Contact
(V-DW11-345)
Quick Reference
vivint.

The Vivint Door/Window Contact (DW11) is a sensor device that can be installed on doors, windows, and other objects in order to monitor open and closed states. The DW11 transmits a signal to the panel when the magnet is moved away from, or close to, the DW11 sensor.

The DW11 device has an external input for NC (Normally Closed) dry contact devices, or it can be used with the supplied magnet directly with the sensor.

The DW11 is also equipped with a cover tamper for additional security.

Programming Instructions
(For more details, go to the Support page at: Vivint.com, and the KIM site at: corp.vivint.com/sites/KIM/)

- **Loop 1:** Use when the external input is used.
- **Loop 2 (default):** Use when the magnet is used directly with the sensor.

Installation Instructions
For internal switch usage:

1. Secure the sensor to the door or window frame, or other object, using the supplied adhesive or screws.
   NOTE: It is recommended to install the sensor on the side that will be moving less (e.g. the doorframe instead of the door).
2. Secure the included magnet adjacent to the sensor on the door, window, or other object using the supplied adhesive or screws.
   NOTE: If necessary, use the supplied magnet spacer to better align the magnet and sensor. Remove the back from the magnet using a small flathead screwdriver and replace it with the spacer back.

For external switch usage:

1. Secure the external contact switch in the desired location.
2. If necessary, drill a hole to allow the external contact wires to reach the desired location of the sensor.
3. Feed the wires through the drilled hole and then through the access hole in the back of the sensor.
4. Connect the wires into the NC dry contact connector.
   NOTE: With solid-core wire you should be able to push the wire directly into the connector. Press the small button on the connector to release the wire.
5. Mount the sensor in the desired location and reattach the sensor cover (store excess wire in the device).

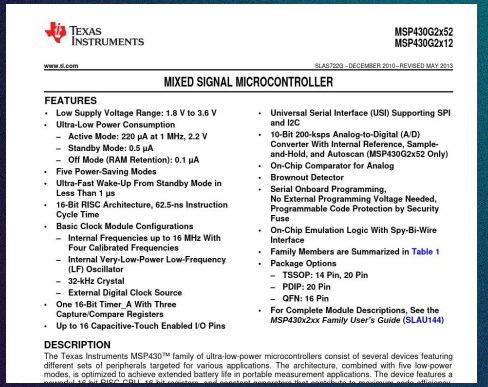
Battery Pull Tab
Tamper Switch
NC dry contact connector
Screw Mounting Features
Access Hole for External Input

Installer Test
Once installed, open and/or close the door or window for which the DW11 is installed so that the sensor will transmit to the Vivint Control Panel while the panel is in installer test mode.

User Test
Open and/or close the door or window for which the DW11 is installed to ensure the sensor is transmitting correctly to the panel. The panel should recognize the state change of the object that is being monitored.

Specifications

Wireless Signal Range	350 feet (106.7 m), open air
Battery	Panasonic CR2032 or equivalent lithium battery
Transmitter Frequency	345 MHz
Code Outputs	Open, Close, Tamper, Low Battery, Loss of Supervision
Supervisory Interval	70 minutes per signal (12 hours for panel to report supervision failure)
Operating Temperature Limits	32° to 120°F (0° to 49°C)
Relative Humidity	5-95% Non-Condensing



Texas Instruments
MSP430G2x52
MSP430G2x12
www.ti.com
SLAS722G – DECEMBER 2010 – REVISED MAY 2013

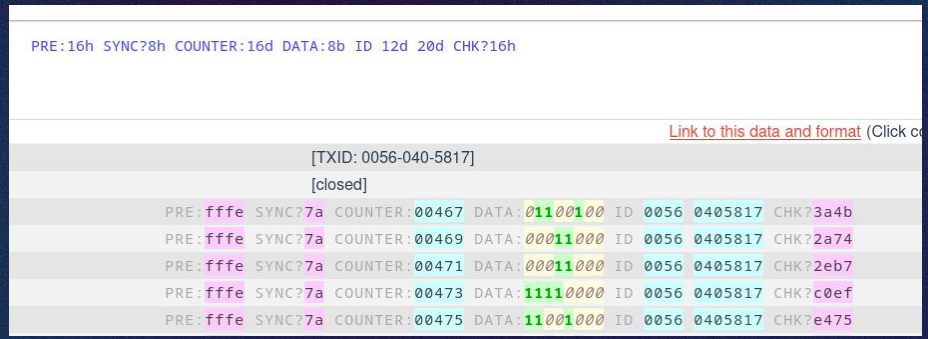
MIXED SIGNAL MICROCONTROLLER

FEATURES

- Low Supply Voltage Range: 1.8 V to 3.6 V
- Ultra-Low Power Consumption
  - Active Mode: 220 µA at 1 MHz, 2.2 V
  - Standby Mode: 0.5 µA
  - Off Mode (RAM Retention): 0.1 µA
- Five Power-Saving Modes
- Ultra-Fast Wake-Up From Standby Mode in Less Than 1 µs
- 16-Bit RISC Architecture, 62.5-ns Instruction Cycle Time
- Basic Clock Module Configurations
  - Internal Frequencies up to 16 MHz With Four Calibrated Frequencies
  - Internal Very-Low-Power Low-Frequency (LF) Oscillator
  - 32-kHz Crystal
  - External Digital Clock Source
- One 16-Bit Timer_A With Three Capture/Compare Registers
- Up to 16 Capacitive-Touch Enabled I/O Pins
- Universal Serial Interface (USI) Supporting SPI and I2C
- 10-Bit 200-ksps Analog-to-Digital (A/D) Converter With Internal Reference, Sample-and-Hold, and Autoscan (MSP430G2x52 Only)
- On-Chip Comparator for Analog
- Brownout Detector
- Serial Onboard Programming, No External Programming Voltage Needed, Programmable Code Protection by Security Fuse
- On-Chip Emulation Logic With Spy-Bi-Wire Interface
- Family Members are Summarized in Table 1
- Package Options
  - TSSOP: 14 Pin, 20 Pin
  - PDIP: 20 Pin
  - QFN: 16 Pin
- For Complete Module Descriptions, See the *MSP430x2xx Family User's Guide* (SLAU144)

DESCRIPTION

The Texas Instruments MSP430™ family of ultra-low-power microcontrollers consist of several devices featuring different sets of peripherals targeted for various applications. The architecture, combined with five low-power modes, is optimized to achieve extended battery life in portable measurement applications. The device features a



```
PRE:16h SYNC?8h COUNTER:16d DATA:8b ID 12d 20d CHK?16h
```

Link to this data and format (Click c

[TXID: 0056-040-5817]
[closed]

```
PRE:fffe SYNC?7a COUNTER:00467 DATA:01100100 ID 0056 0405817 CHK?3a4b
PRE:fffe SYNC?7a COUNTER:00469 DATA:00011000 ID 0056 0405817 CHK?2a74
PRE:fffe SYNC?7a COUNTER:00471 DATA:00011000 ID 0056 0405817 CHK?2eb7
PRE:fffe SYNC?7a COUNTER:00473 DATA:11110000 ID 0056 0405817 CHK?c0ef
PRE:fffe SYNC?7a COUNTER:00475 DATA:11001000 ID 0056 0405817 CHK?e475
```

Where to start? There be dragons ahead!

Gather Context

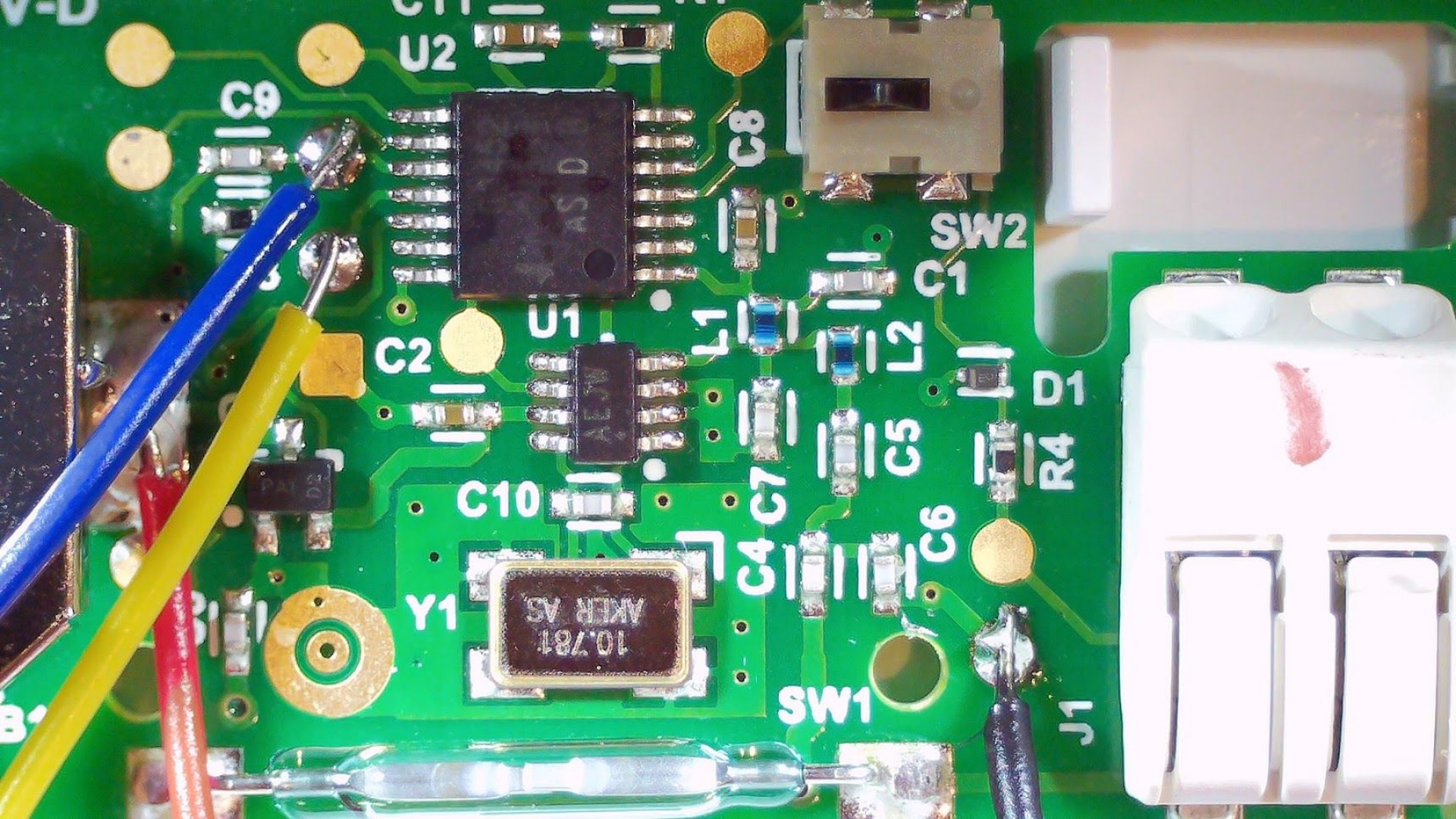
Try to open the device and identify the chip. Often, the chip is written on it

Identify the other chips

MCU Datasheets tell you almost everything you need to know about the chip

Start collecting information about what you know

Get the firmware... and all the other data on the chip


U2
C9
C8
SW2
C1
U1
L1
L2
C2
D1
C5
R4
C10
C7
C6
C4
Y1
10.787
AKER AS
SW1
J1

Now we know some pins

- AEJW - MAX7044 OOK Encoder

- 10.781MHz Oscillator

- Reverse voltage protection

- We now know the output pin on the MSP430 that outputs the pre-OOK encoded data
- Tamper pin
- Contact Pin
- Reed Pin

Look at similar devices, duh.

The PIR2 and GB2 have a debug port

The PIR2 also has an MSP430 that has additional math peripherals, more PSRAM, and more code space simplifying the reverse engineering due to fewer optimizations

Compare the firmwares, because they both probably use the same encryption



Tiers of Embedded Firmware

Monolith -> RTOS -> Embedded Linux

Monolith: Completely statically compiled firmware where everything is included in the firmware image. Granular control of everything on chip.

RTOS: Real Time Operating System, esp32s use this even when they used the Arduino IDE. Functionality divided into ‘Task’ with managed heap and scheduling.

Embedded Linux: Full blown linux, often with minimalistic programs like Busybox

My humble V-DW11-345 is a **monolithic** firmware

Time for Dragons!



Don’t worry, is fren

What is Ghidra?

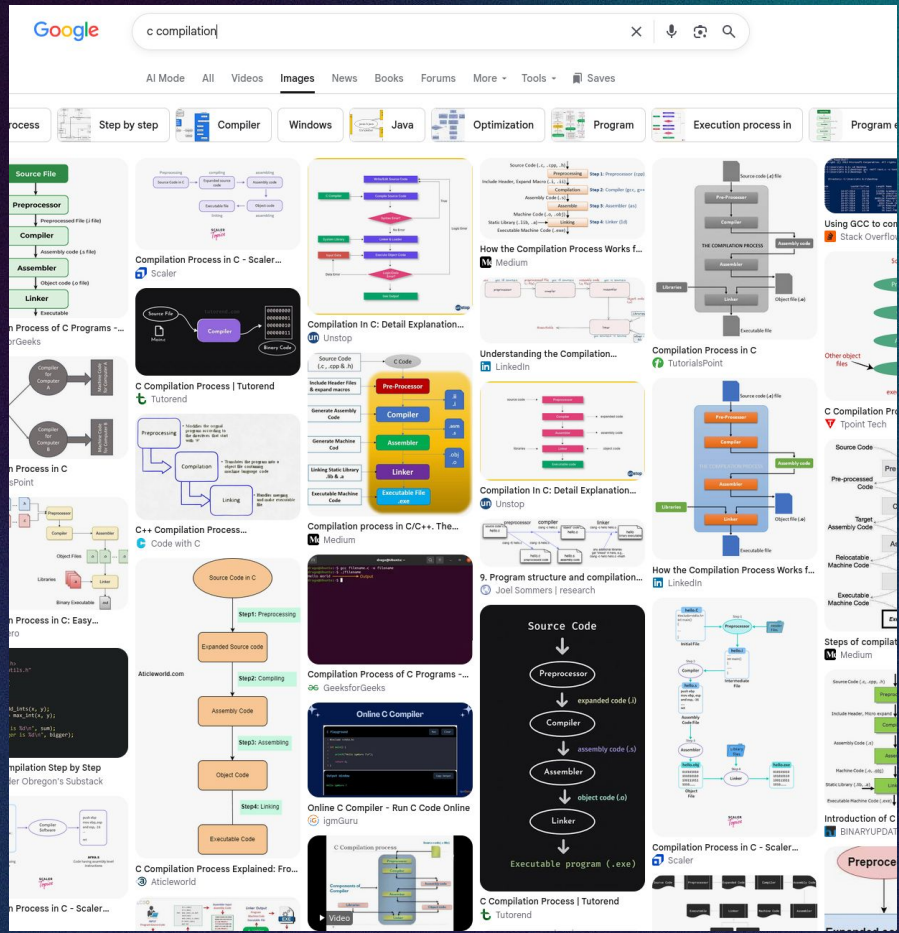
To understand Ghidra, you need to understand compilation

Compilation:

C code ->
Preprocessing ->
Compiling ->
Assembling ->
Linking ->
Machine code (1's and 0's)

Ghidra:

Machine code ->
P-Code ->
Approximate C code



Why is it “Approximate”?

Things you lose:

1. Variable Names
2. Comments
3. Function Names
4. Enums
5. #defines
6. Constants
7. Structs and Classes

Optimizations can nuke

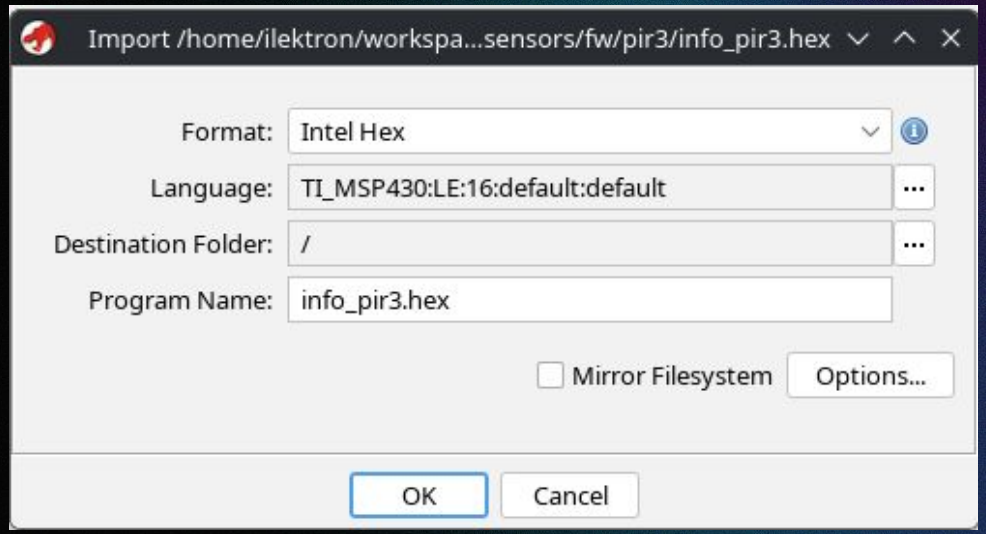
1. Some function calls
2. Some variables
3. C code flow
4. And more

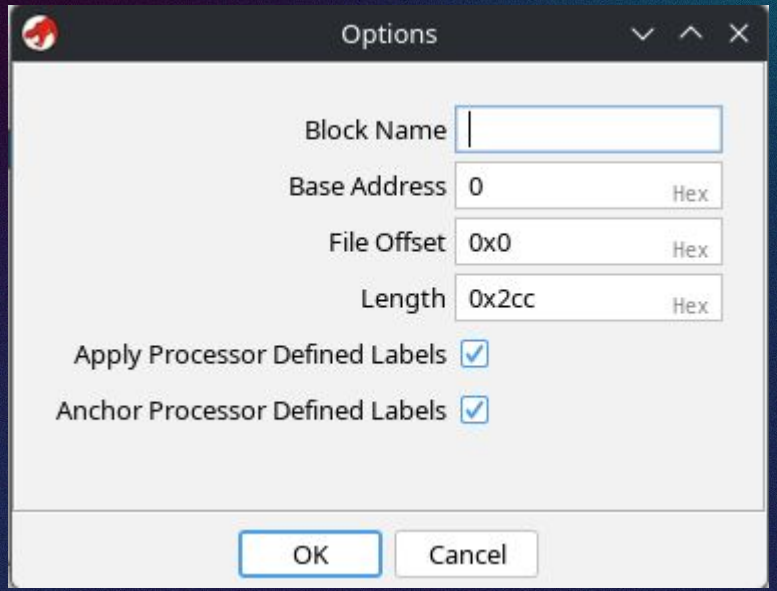


Importing our Firmware

Ghidra needs to know

1. Format
    a. Hex vs Raw Binary
2. Language

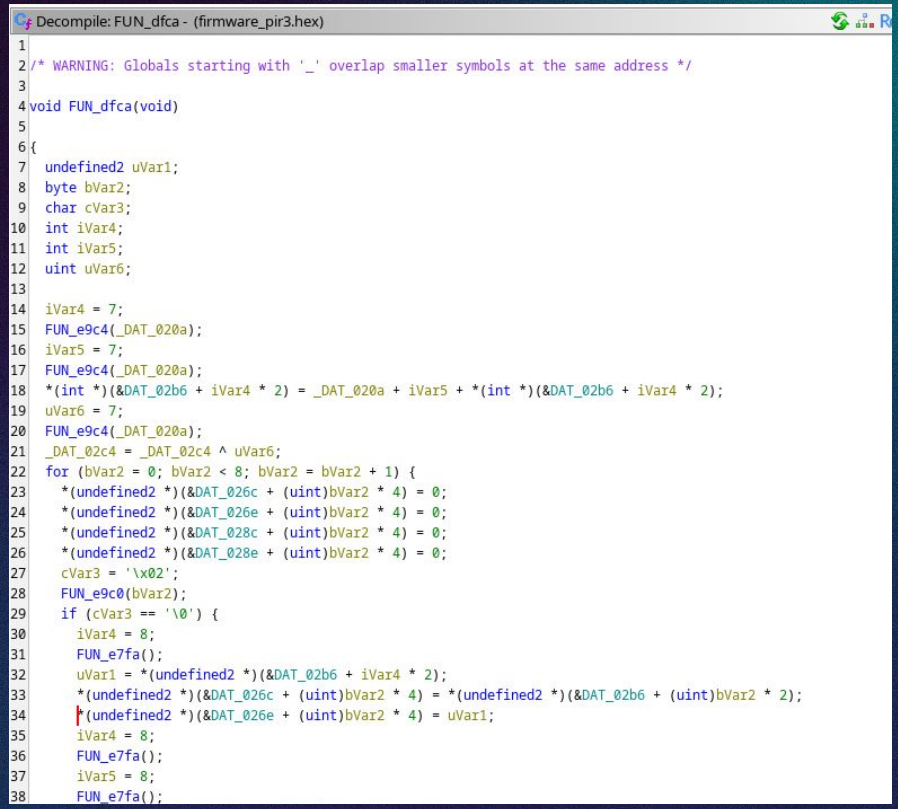




General Tips for starting with Ghidra

Much like that job you had early (or later) in your career when you open the code base only to find out all variables are 6 letters or less, 1000+ line recursive functions, and no comments or documentation.

- Define memory areas and addresses
- Find entry points
- Start giving things names (even if not sure)
- Add comments
- Define functions
  - Fix function signatures
  - Commit the signatures
- Set variable types
  - Add enums and struct
- Define arrays



Specific to embedded hardware

- **Memory Mapped Peripherals**
  - Datasheet has information about memory map, useful for extracting data from chip
- **Interrupts**
  - Interrupt table
  - Define Interrupt Handlers
- **Check pin definitions**
  - Compare with PCB
- **Review EABI**
  - If you need to look at assembly and fix function signatures, the EABI can be very useful
  - Toolchains can change the EABI over time and are MCU specific
  - C standard defines behaviors, not implementations

Program Trees

firmware_dw12.hex
RAM

Program Tree ×

Symbol Tr...

Imports
Exports
Functions
Labels
Classes
Namespaces

Filter:

Data Type Manager

Data Types
BuiltInTypes
firmware_dw12.hex
generic_clib

Listing: firmware_dw12.hex

```
                    //
                    // RAM
                    // Generated by Intel Hex
                    // RAM:e000-RAM:ffff
                    //
e000 4d             ??          4Dh    M
e001 d3             ??          D3h
e002 34             ??          34h    4
e003 4d             ??          4Dh    M

                    DAT_e004                                    XREF[1]:     FUN_f36a:f3e8(R)
e004 d3 34          undefined2  34D3h

                    DAT_e006                                    XREF[1]:     FUN_f36a:f3ec(R)
e006 4d d3          undefined2  D34Dh

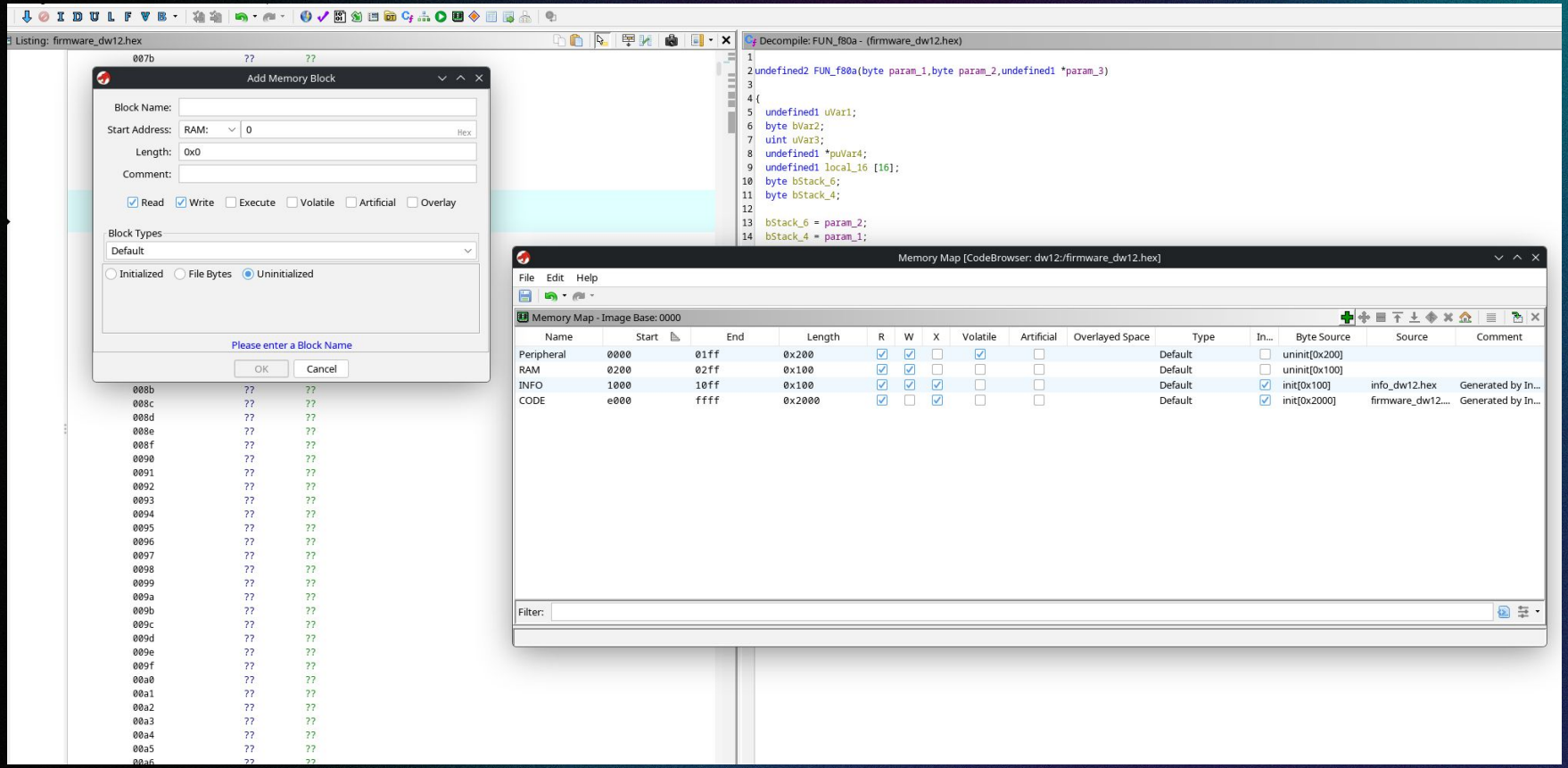
                    DAT_e008                                    XREF[1]:     FUN_f36a:f3e8(R)
e008 34 4d          undefined2  4D34h
e00a d3             ??          D3h
e00b 34             ??          34h    4
e00c 4d             ??          4Dh    M
e00d d3             ??          D3h
e00e 34             ??          34h    4
e00f 4d             ??          4Dh    M
e010 d3             ??          D3h
e011 34             ??          34h    4
e012 4d             ??          4Dh    M
e013 d3             ??          D3h
e014 34             ??          34h    4
e015 4d             ??          4Dh    M
e016 d3             ??          D3h
e017 34             ??          34h    4
e018 4d             ??          4Dh    M
e019 d3             ??          D3h
e01a 34             ??          34h    4
e01b 4d             ??          4Dh    M
e01c d3             ??          D3h
e01d 34             ??          34h    4
e01e 4d             ??          4Dh    M
e01f d3             ??          D3h

                    DAT_e020                                    XREF[1]:     e510(*)
e020 02             ??          02h
e021 00             ??          00h
e022 00             ??          00h
e023 00             ??          00h
e024 22             ??          22h    "
```

Decompile: UndefinedFunction_e252 - (firmware_dw12.hex)

```
1
2 /* WARNING: This function may have set the stack pointer */
3 /* WARNING: Globals starting with '_' overlap smaller symbols at the same address
4
5 undefined2 UndefinedFunction_e252(void)
6
7 {
8   bool bVar1;
9   byte bVar2;
10  undefined2 uVar3;
11  undefined *puVar4;
12  undefined2 uVar5;
13  undefined2 in_R15;
14
15  _DAT_02fe = (undefined *)0xe262;
16  FUN_fdee(&DAT_0218,0xb2);
17  uVar3 = 0x200;
18  puVar4 = &UNK_e23c;
19  uVar5 = 0x17;
20  _DAT_02fe = (undefined *)0xe272;
21  thunk_FUN_fda6();
22  _DAT_02fe = (undefined *)0xe276;
23  FUN_ee16();
24  _DAT_02fe = (undefined *)0xe27a;
25  thunk_FUN_fd92();
26  _DAT_0228 = _DAT_0228 + 0x31d;
27  _DAT_021e = _DAT_021e + 1;
28  _DAT_02f6 = 58000;
29  _DAT_02f8 = uVar5;
30  _DAT_02fa = in_R15;
31  _DAT_02fc = uVar3;
32  _DAT_02fe = puVar4;
33  FUN_fc34();
34  bVar2 = DAT_0027;
35  DAT_0027 = bVar2 & 0xb3;
36  bVar2 = P1OUT;
37  P1OUT = ~DAT_02b7 & bVar2;
38  bVar2 = P1DIR;
39  P1DIR = DAT_02b7 | bVar2;
40  bVar2 = P2DIR;
41  P2DIR = bVar2 & 0x7f;
42  bVar2 = P2OUT;
43  P2OUT = bVar2 | 0x80;
44  bVar2 = DAT_002f;
45  DAT_002f = bVar2 | 0x80;
46  bVar2 = P2IN;
47  DAT_02b8 = bVar2 & 0x80;
```

Defining Memory Address



Graph Navigation Search Select Tools Window Help

Listing: firmware_dw12.hex

```
                                                        e784(W), e7be(W), e806(W),
                                                        e82e(W), e838(W), e8b4(W),
                                                        e904(W), e952(W), [more]
            02f6                undefined2 ??

                        DAT_02f8                        XREF[2]:    e280(W), e978(R)
            02f8                undefined2 ??

                        DAT_02fa                        XREF[2]:    e27e(W), e97a(R)
            02fa                undefined2 ??

                        DAT_02fc                        XREF[2]:    e27c(W), e97c(R)
            02fc                undefined2 ??

                        DAT_02fe                        XREF[6]:    e25e(W), e26e(W), e272(W),
                                                                    e276(W), e27a(W), e97e(R)
            02fe                undefined2 ??
            .....
                        //
                        // INFO
                        // Generated by Intel Hex
                        // RAM:1000-RAM:10ff
                        //

                        DAT_1000                        XREF[3]:    FUN_f80a:f852(W),
                                                                    FUN_f80a:f862(W),
                                                                    FUN_fbd0:fbe0(*)
            1000 5c             undefined1 5Ch

                        DAT_1001                        XREF[1]:    FUN_f80a:f862(W)
            1001 3f             undefined1 3Fh
            1002 b7             ??         B7h
            1003 06             ??         06h
            1004 1b             ??         1Bh
            1005 05             ??         05h
            1006 ff             ??         FFh
            1007 ff             ??         FFh
            1008 01             ??         01h
            1009 05             ??         05h
            100a ce             ??         CEh
            100b 4f             ??         4Fh    O
            100c 0c             ??         0Ch
            100d ff             ??         FFh
            100e ff             ??         FFh
            100f ff             ??         FFh
            1010 ff             ??         FFh
            1011 ff             ??         FFh
            1012 ff             ??         FFh
            1013 ff             ??         FFh
```

Decompile: FUN_f80a - (firmware_dw12.hex)

```
1
2 undefined2 FUN_f80a(byte param_1,byte param_2,undefined1 *param_3)
3
4 {
5   undefined1 uVar1;
6   byte bVar2;
7   uint uVar3;
8   undefined1 *puVar4;
9   undefined1 local_16 [16];
10  byte bStack_6;
11  byte bStack_4;
12
13  bStack_6 = param_2;
14  bStack_4 = param_1;
15  FUN_fbd0(0,0x10,local_16);
16  for (uVar3 = 0; (byte)uVar3 < bStack_6; uVar3 = (uint)(byte)((byte)uVar3 + 1)) {
17    uVar1 = *param_3;
18    param_3 = param_3 + 1;
19    local_16[bStack_4 + uVar3] = uVar1;
20  }
21  puVar4 = &DAT_1020;
22  FCTL3 = 0xa500;
23  FCTL1 = 0xa502;
24  DAT_1000 = 0;
25  FCTL1 = 0xa540;
26  uVar3 = 0;
27  do {
28    *puVar4 = local_16[uVar3];
29    puVar4 = puVar4 + 1;
30    bVar2 = (char)uVar3 + 1;
31    uVar3 = (uint)bVar2;
32  } while (bVar2 < 0x10);
33  FCTL1 = 0xa500;
34  FCTL3 = 0xa510;
35  return 0;
36 }
37
```

Define entry points – Know your MCU

Interrupt Vector Addresses

The interrupt vectors and the power-up starting address are located in the address range 0FFFFh to 0FFC0h. The vector contains the 16-bit address of the appropriate interrupt handler instruction sequence.

If the reset vector (located at address 0FFFEh) contains 0FFFFh (for example, if flash is not programmed) the CPU goes into LPM4 immediately after power-up.

Table 5. Interrupt Sources, Flags, and Vectors

INTERRUPT SOURCE	INTERRUPT FLAG	SYSTEM INTERRUPT	WORD ADDRESS	PRIORITY
Power-Up External Reset Watchdog Timer+ Flash key violation PC out-of-range(1)	PORIFG RSTIFG WDTIFG KEYV(2)	Reset	0FFFEh	31, highest
NMI Oscillator fault Flash memory access violation	NMIIFG OFIFG ACCVIFG(2)(3)	(non)-maskable (non)-maskable (non)-maskable	0FFFCh	30
			0FFFAh	29
			0FFF8h	28
Comparator_A+	CAIFG(4)	maskable	0FFF6h	27
Watchdog Timer+	WDTIFG	maskable	0FFF4h	26
Timer0_A3	TACCR0 CCIFG(4)	maskable	0FFF2h	25
Timer0_A3	TACCR2 TACCR1 CCIFG. TAIFG(2)(4)	maskable	0FFF0h	24
			0FFEEh	23
			0FFECh	22
ADC10(5)	ADC10IFG(4)(5)	maskable	0FFEAh	21
USI	USIIFG, USISTTIFG(2)(4)	maskable	0FFE8h	20
I/O Port P2 (up to eight flags)	P2IFG.0 to P2IFG.7(2)(4)	maskable	0FFE6h	19
I/O Port P1 (up to eight flags)	P1IFG.0 to P1IFG.7(2)(4)	maskable	0FFE4h	18
			0FFE2h	17
			0FFE0h	16
See (6)			0FFDEh to 0FFC0h	15 to 0, lowest

(1) A reset is generated if the CPU tries to fetch instructions from within the module register memory address range (0h to 01FFh) or from

Listing: firmware_dw12.hex

```
                        INT_FFDC                                XREF[1]:     Entry Point(*)
ffdc ff ff              addr        ffff

                        INT_FFDE                                XREF[1]:     Entry Point(*)
ffde ff ff              addr        ffff

                        INT_FFE0                                XREF[1]:     Entry Point(*)
ffe0 ff ff              addr        ffff

                        INT_FFE2                                XREF[1]:     Entry Point(*)
ffe2 ff ff              addr        ffff

                        INT_FFE4                                XREF[1]:     Entry Point(*)
ffe4 ff ff              addr        ffff

                        INT_FFE6                                XREF[1]:     Entry Point(*)
ffe6 ff ff              addr        ffff

                        INT_FFE8                                XREF[1]:     Entry Point(*)
ffe8 ff ff              addr        ffff

                        INT_FFEA                                XREF[1]:     Entry Point(*)
ffea 34 ff              addr        LAB_ff34

                        INT_FFEC                                XREF[1]:     Entry Point(*)
ffec ff ff              addr        ffff

                        INT_FFEE                                XREF[1]:     Entry Point(*)
ffee ff ff              addr        ffff

                        INT_FFF0                                XREF[1]:     Entry Point(*)
fff0 4a f1              addr        LAB_f14a

                        INT_FFF2                                XREF[1]:     Entry Point(*)
fff2 7a e2              addr        LAB_e27a

                        INT_FFF4                                XREF[1]:     Entry Point(*)
fff4 ff ff              addr        ffff

                        INT_FFF6                                XREF[1]:     Entry Point(*)
fff6 54 fc              addr        LAB_fc54

                        INT_FFF8                                XREF[1]:     Entry Point(*)
fff8 ff ff              addr        ffff

                        USER_NMI                                XREF[1]:     Entry Point(*)
fffa ff ff              addr        ffff

                        SYSTEM_NMI                              XREF[1]:     Entry Point(*)
fffc 7e fd              addr        LAB_fd7e

                        RESET                                   XREF[1]:     Entry Point(*)
fffe 52 e2              addr        LAB_e252
```

Decompile: UndefinedFunction_e252 - (firmware_dw12.hex)

```
1
2  /* WARNING: This function may have set the stack pointer */
3  /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
4
5  undefined2 UndefinedFunction_e252(void)
6
7  {
8    bool bVar1;
9    byte bVar2;
10   undefined2 uVar3;
11   undefined2 uVar4;
12   undefined2 uVar5;
13   undefined2 in_R15;
14
15   _DAT_02fe = 0xe262;
16   FUN_fdee(&DAT_0218,0xb2);
17   uVar3 = 0x200;
18   uVar4 = 0xe23c;
19   uVar5 = 0x17;
20   _DAT_02fe = 0xe272;
21   thunk_FUN_fda6();
22   _DAT_02fe = 0xe276;
23   FUN_ee16();
24   _DAT_02fe = 0xe27a;
25   thunk_FUN_fd92();
26   _DAT_0228 = _DAT_0228 + 0x31d;
27   _DAT_021e = _DAT_021e + 1;
28   _DAT_02f6 = 58000;
29   _DAT_02f8 = uVar5;
30   _DAT_02fa = in_R15;
31   _DAT_02fc = uVar3;
32   _DAT_02fe = uVar4;
33   FUN_fc34();
34   bVar2 = DAT_0027;
35   DAT_0027 = bVar2 & 0xb3;
36   bVar2 = P1OUT;
37   P1OUT = ~DAT_02b7 & bVar2;
38   bVar2 = P1DIR;
39   P1DIR = DAT_02b7 | bVar2;
40   bVar2 = P2DIR;
41   P2DIR = bVar2 & 0x7f;
42   bVar2 = P2OUT;
43   P2OUT = bVar2 | 0x80;
44   bVar2 = DAT_002f;
45   DAT_002f = bVar2 | 0x80;
46   bVar2 = P2IN;
47   DAT_02b8 = bVar2 & 0x80;
48   bVar2 = DAT_002f;
49   DAT_002f = bVar2 & 0x7f;
50   bVar2 = P2OUT;
51   P2OUT = ~DAT_02b8 & bVar2;
52   bVar2 = P2DIR;
53   P2DIR = DAT_02b8 | bVar2;
54   if ((DAT_02c7 & 2) == 0) {
55     DAT_02b7 = DAT_02b7 | 0x40;
```

```
ffde ff ff          addr        ffff

                    INT_FFE0                                XREF[1]:    Entry Point(*)
ffe0 ff ff          addr        ffff

                    INT_FFE2                                XREF[1]:    Entry Point(*)
ffe2 ff ff          addr        ffff

                    INT_FFE4                                XREF[1]:    Entry Point(*)
ffe4 ff ff          addr        ffff

                    INT_FFE6                                XREF[1]:    Entry Point(*)
ffe6 ff ff          addr        ffff

                    INT_FFE8                                XREF[1]:    Entry Point(*)
ffe8 ff ff          addr        ffff

                    INT_FFEA                                XREF[1]:    Entry Point(*)
ffea 34 ff          addr        ADC10IFG_handler

                    INT_FFEC                                XREF[1]:    Entry Point(*)
ffec ff ff          addr        ffff

                    INT_FFEE                                XREF[1]:    Entry Point(*)
ffee ff ff          addr        ffff

                    INT_FFF0                                XREF[1]:    Entry Point(*)
fff0 4a f1          addr        TACCR2_TACCR1_CCIFG_TAIFG_handler

                    INT_FFF2                                XREF[1]:    Entry Point(*)
fff2 7a e2          addr        TACCR0_CCIFG_handler

                    INT_FFF4                                XREF[1]:    Entry Point(*)
fff4 ff ff          addr        ffff

                    INT_FFF6                                XREF[1]:    Entry Point(*)
fff6 54 fc          addr        CAIFG_handler

                    INT_FFF8                                XREF[1]:    Entry Point(*)
fff8 ff ff          addr        ffff

                    USER_NMI                                XREF[1]:    Entry Point(*)
fffa ff ff          addr        ffff

                    SYSTEM_NMI                              XREF[1]:    Entry Point(*)
fffc 7e fd          addr        nmi_handler

                    RESET                                   XREF[1]:    Entry Point(*)
fffe 52 e2          addr        __init
```

```
byte bVar2;

iVar1 = TAIV;
if (iVar1 != 2) {
  return;
}
if (DAT_020e <= DAT_0208) {
  FUN_ff3c();
  if (DAT_02b4 != 0xde) {
    return;
  }
  DAT_021c = DAT_021c + 1;
  if (0x1d5d < DAT_021c) {
    DAT_021c = 0;
    DAT_021a = DAT_021a + 1;
  }
  if (DAT_021a < 0x14) {
    return;
  }
  FUN_ff44();
  DAT_02c7 = DAT_02c7 & 0xfd;
  FUN_ff0c();
  DAT_020c = 0xfd;
  DAT_020d = 0x1d;
  DAT_02bd = 9;
  DAT_0278 = 0;
  DAT_022e = 0;
  FUN_fe54();
  return;
}
if (DAT_02b4 < 10) {
  if (DAT_02b4 == 0) {
    bVar2 = P2OUT;
    P2OUT = bVar2 | 0x40;
    bVar2 = P2OUT;
    P2OUT = bVar2 | 0x40;
    FUN_ff3c();
    FUN_ff3c();
  }
  DAT_02b4 = DAT_02b4 + 1;
  return;
}
if ((DAT_020c < 0x17) || (DAT_020c == 0xfd)) {
  DAT_028e = (*(byte *)((int)&DAT_0294 + (uint)DAT_0208) & DAT_028d) != 0;
  bVar2 = P2OUT;
  if ((bool)DAT_028e) {
    bVar2 = bVar2 | 0x40;
  }
  else {
    bVar2 = bVar2 & 0xbf;
```

Console - Scripting

Name stuff & add comments - better to make guesses

Decompile: FUN_fc34 - (firmware_dw12.hex)

```
1
2 void FUN_fc34(void)
3
4 {
5   byte bVar1;
6
7   bVar1 = P1DIR;
8   P1DIR = bVar1 & 0xb3;
9   bVar1 = P1OUT;
10  P1OUT = bVar1 | 0x4c;
11  bVar1 = DAT_0027;
12  DAT_0027 = bVar1 | 0x4c;
13  bVar1 = P1IN;
14  DAT_02b7 = bVar1 & 0x4c;
15  return;
16 }
17
```

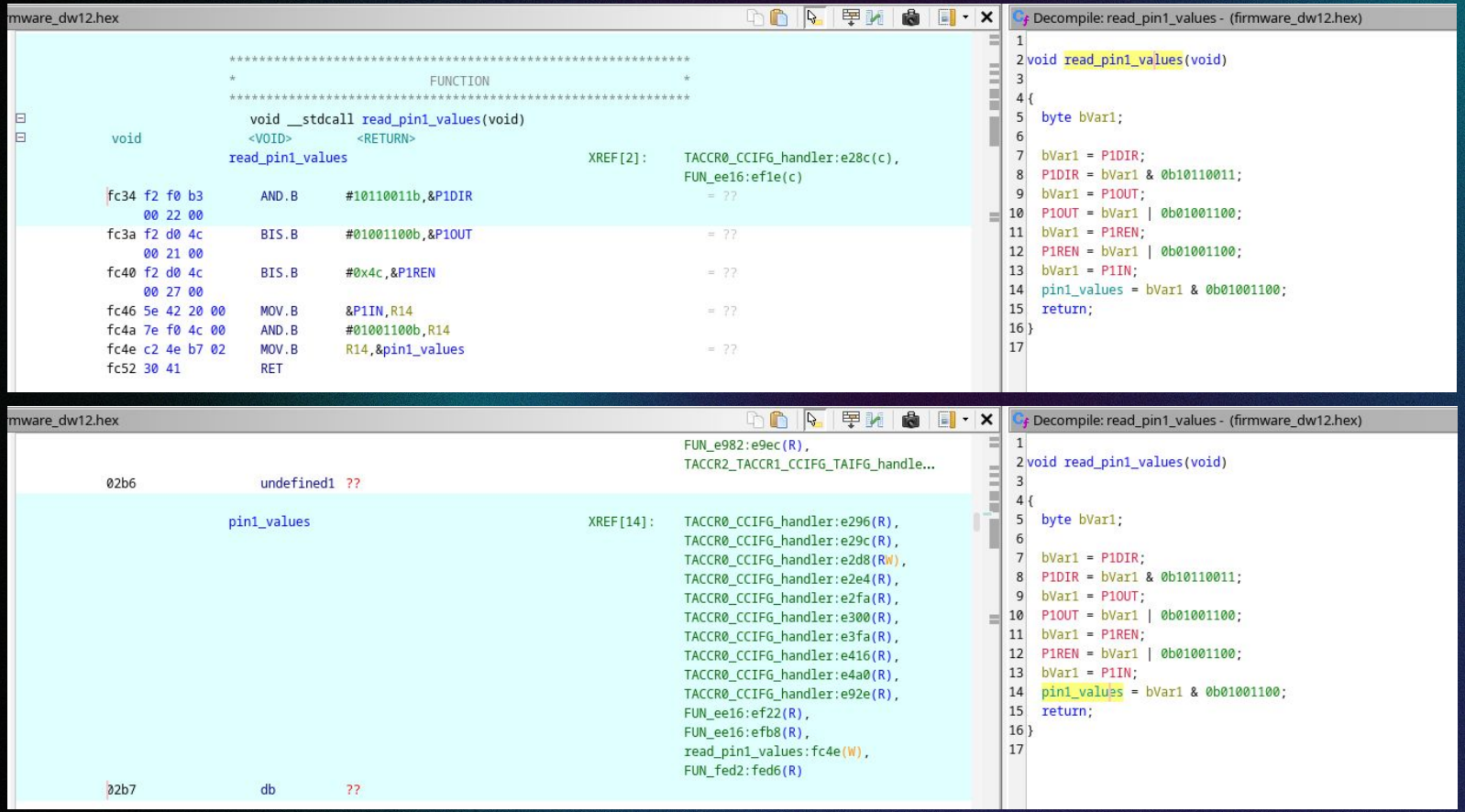
Decompile: FUN_fc34 - (firmware_dw12.hex)

```
1
2 void FUN_fc34(void)
3
4 {
5   byte bVar1;
6
7   bVar1 = P1DIR;
8   P1DIR = bVar1 & 0b10110011;
9   bVar1 = P1OUT;
10  P1OUT = bVar1 | 0b01001100;
11  bVar1 = P1REN;
12  P1REN = bVar1 | 0b01001100;
13  bVar1 = P1IN;
14  pin1_values = bVar1 & 0b01001100;
15  return;
16 }
17
```

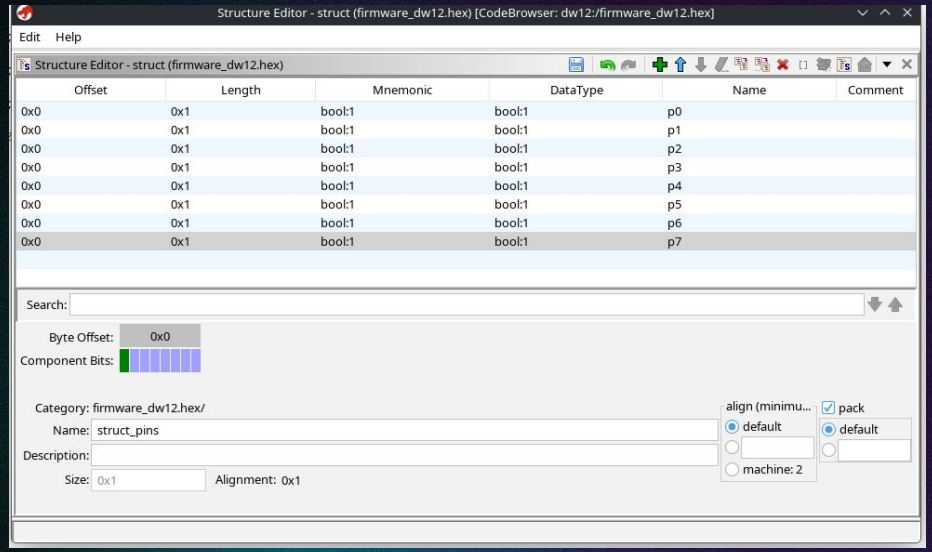
Decompile: read_pin1_values - (firmware_dw12.hex)

```
1
2 void read_pin1_values(void)
3
4 {
5   byte bVar1;
6
7   bVar1 = P1DIR;
8   P1DIR = bVar1 & 0b10110011;
9   bVar1 = P1OUT;
10  P1OUT = bVar1 | 0b01001100;
11  bVar1 = P1REN;
12  P1REN = bVar1 | 0b01001100;
13  bVar1 = P1IN;
14  pin1_values = bVar1 & 0b01001100;
15  return;
16 }
17
```

Use the tools to find where things are used

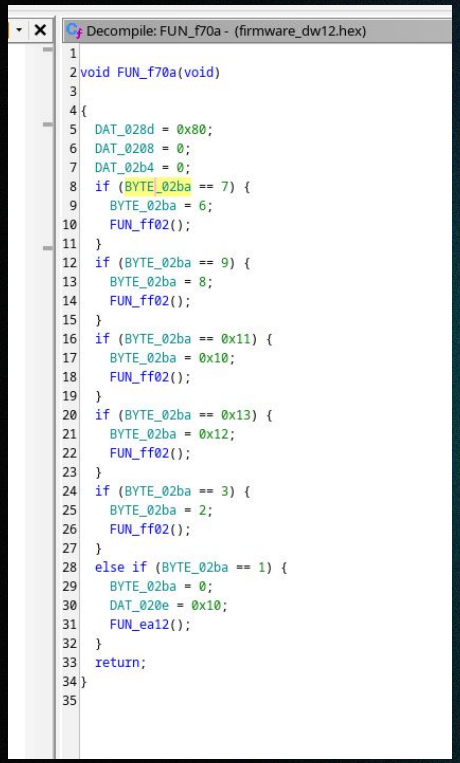


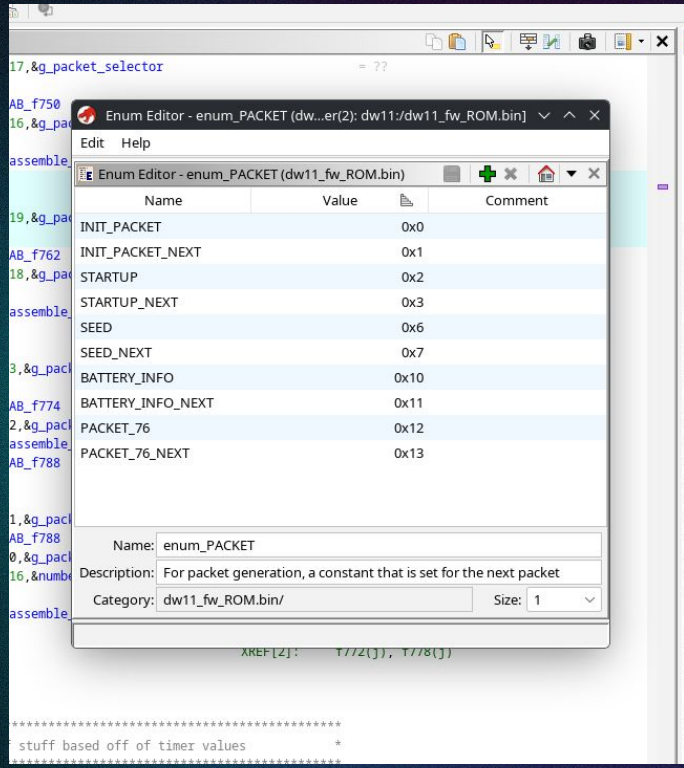
Defining structs and enums

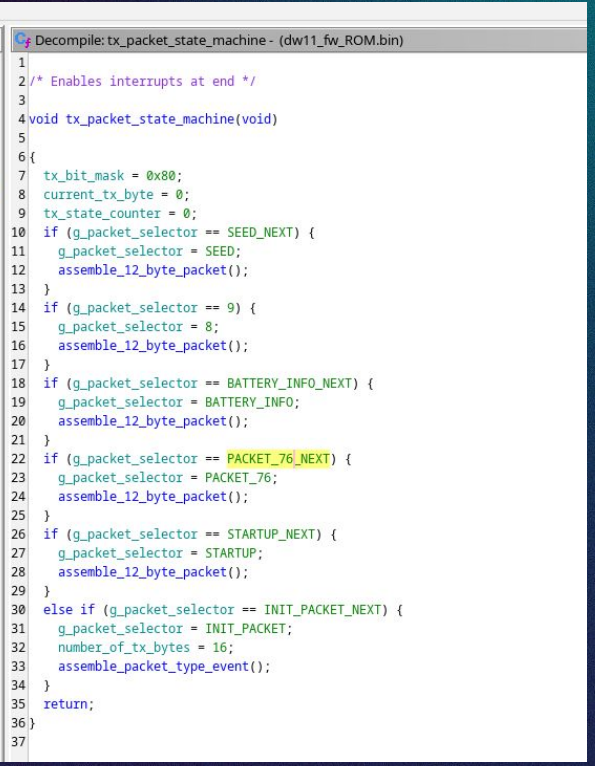



```
if ((((pin1_values.p2) && (pin1_values.p3)) && ((char)DAT_02b8 < 0)) && (DAT_022c <= DAT_0202)
    ) {
  sVar2 = P1OUT;
  P1OUT = (struct_pins)((byte)sVar2 & 0x7f);
  DAT_022a = 0;
}
```

Defining structs and enums







EABI – Embedded Application Binary Interface

1.2 Scope

Figure 1-1 shows the components of the ABI and their relationship. We will briefly describe the components, beginning with the lower part of the diagram and moving upward, and provide references to the appropriate chapter of this ABI specification.

The components in the bottom area relate to object-level interoperability.

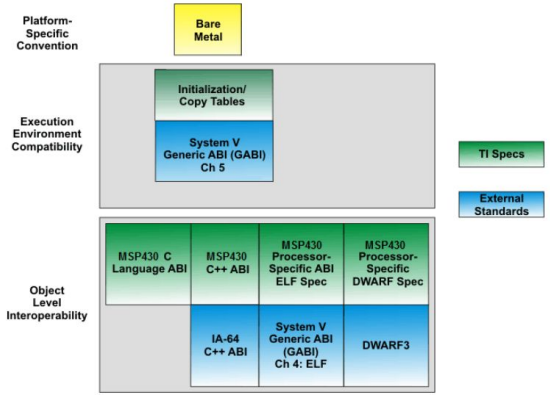


Figure 1-1. Parts of the ABI Specification

The **C Language ABI** (Chapter 2, Chapter 3, Chapter 4, Chapter 5, Chapter 6 and Chapter 7) specifies function calling conventions, data type representations, addressing conventions, and the interface to the C run-time library.

The **C++ ABI** (Chapter 8) specifies how the C++ language is implemented; this includes details about virtual function tables, name mangling, how constructors are called, and the exception handling mechanism (Chapter 9). The MSP430 C++ ABI is based on the prevalent IA-64 (Itanium) C++ ABI.

The **DWARF** component (Chapter 10) specifies the representation of object-level debug information. The base standard is the DWARF3 standard. This specification details processor-specific extensions.

The **ELF** component (Chapter 11) specifies the representation of object files. This specification extends the System V ABI specification with processor specific information.

Build Attributes (Chapter 13) refer to a means of encoding into an object file various parameters that affect inter-object compatibility, such as target device assumptions, memory models, or ABI variants. Toolchains can use build attributes to prevent incompatible object files from being combined or loaded.

Copyright © 2022 Texas Instruments Incorporated

Fixing Function Signatures - can fix a lot of stuff

```
f0b2 f2 90 07 00 bd 02   CMP.B   #0x7,&DAT_02bd                 = ??
f0b8 0e 20               JNE     LAB_f0d6
f0ba 1c 42 78 02         MOV.W   &DAT_0278,R12                 = ??
f0be 3e 40 07 00         MOV.W   #0x7,R14
f0c2 b0 12 ea fc         CALL    #FUN_fcea                     undefined FUN_fcea()
f0c6 82 4c 02 02         MOV.W   R12,&DAT_0202                 = ??
f0ca 1c 42 02 02         MOV.W   &DAT_0202,R12                 = ??
f0ce b0 12 ca fc         CALL    #FUN_fcca                     undefined FUN_fcca()
f0d2 f2 42 bd 02         MOV.B   #8,&DAT_02bd                  = ??
```

```
141   }
142   if (((DAT_02c7 & 2) != 0) && (DAT_02bd == '\a')) {
143     uVar2 = DAT_0278;
144     FUN_fcea();
145     DAT_0202 = uVar2;
146     FUN_fcca();
147     DAT_02bd = '\b';
148   }
149   if (((DAT_0211 == 2) && (DAT_020e <= DAT_0208)) && (DAT_02b5 == '\0')) {
150     bVar1 = BCSCTL1;
```

Listing: firmware_dw12.hex

```
                        *************************************************************
                        *                          FUNCTION                         *
                        *************************************************************
                          undefined FUN_fcea()
          undefined       <UNASSIGNED>   <RETURN>
                        FUN_fcea                                XREF[5]:   FUN_ee16:f0c2(c),
                                                                           FUN_ee16:f12c(c),
                                                                           FUN_fa90:faa8(c),
                                                                           FUN_fb7e:fb9e(c),
                                                                           FUN_ff2a:ff30(c)
          fcea 0f 4e        MOV.W   R14,R15
          fcec 0d 4c        MOV.W   R12,R13
          fcee 0e 43        MOV.W   #0,R14
          fcf0 1c 43        MOV.W   #1,R12

                        LAB_fcf2                                XREF[1]:   fcfe(j)
          fcf2 0d 5d        RLA.W   R13
          fcf4 0e 6e        RLC.W   R14
          fcf6 0e 9f        CMP.W   R15,R14
          fcf8 01 28        JNC     LAB_fcfc
          fcfa 0e 8f        SUB.W   R15,R14

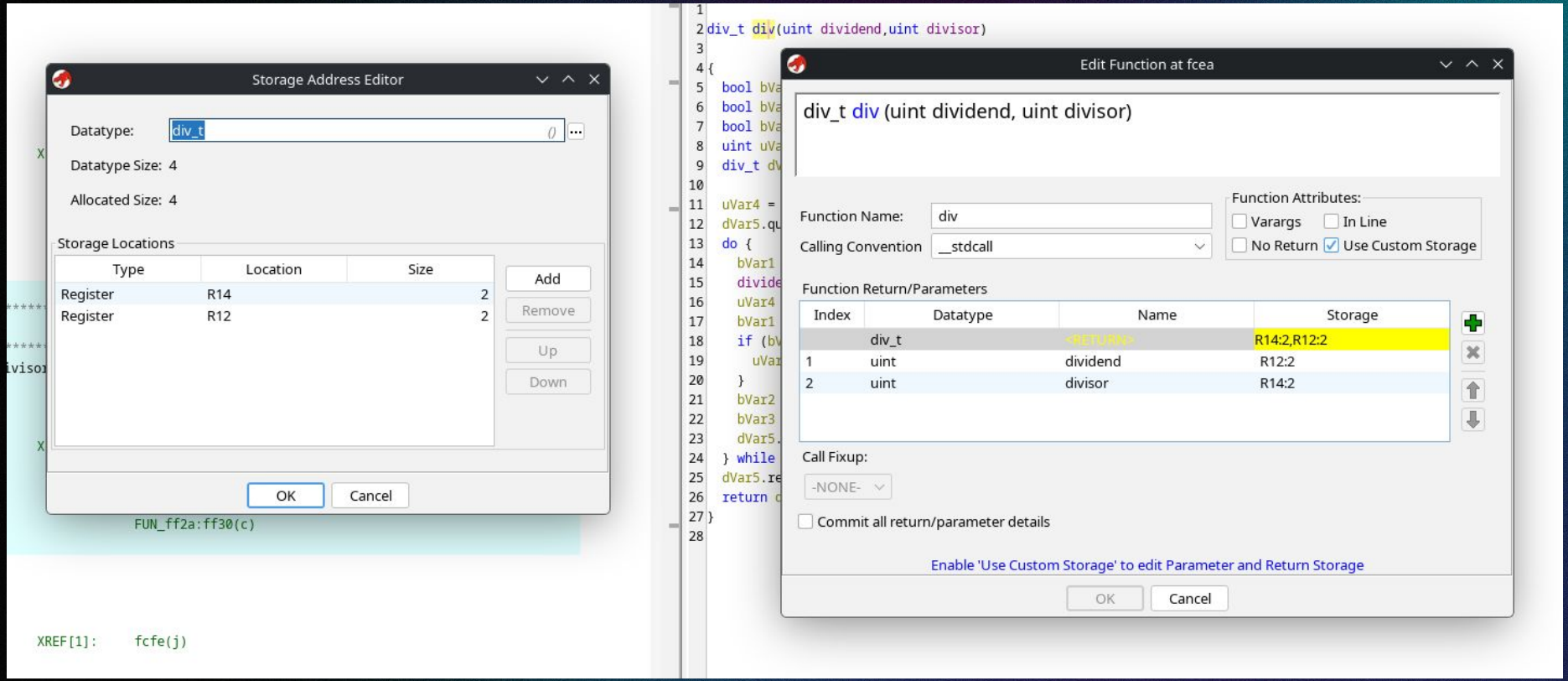
                        LAB_fcfc                                XREF[1]:   fcf8(j)
          fcfc 0c 6c        RLC.W   R12
          fcfe f9 2b        JNC     LAB_fcf2
          fd00 30 41        RET

                        *************************************************************
```

Decompile: FUN_fcea - (firmware_dw12.hex)

```
1
2 void FUN_fcea(uint param_1,undefined2 param_2,uint param_3)
3
4 {
5   bool bVar1;
6   bool bVar2;
7   bool bVar3;
8   uint uVar4;
9   uint uVar5;
10
11  uVar5 = 0;
12  uVar4 = 1;
13  do {
14    bVar1 = CARRY2(param_1,param_1);
15    param_1 = param_1 * 2;
16    uVar5 = uVar5 * 2 + (uint)bVar1;
17    bVar1 = param_3 <= uVar5;
18    if (bVar1) {
19      uVar5 = uVar5 - param_3;
20    }
21    bVar2 = CARRY2(uVar4,(uint)bVar1);
22    bVar3 = CARRY2(uVar4,uVar4 + bVar1);
23    uVar4 = uVar4 * 2 + (uint)bVar1;
24  } while (!bVar2 && !bVar3);
25  return;
26 }
27
```

Fixing Function Signatures - can fix a lot of stuff



Fixing Function Signatures - can fix a lot of stuff

mware_dw12.hex

```
                        *************************************************************
                        *                          FUNCTION                         *
                        *************************************************************
                           div_t __stdcall div(uint dividend, uint divisor)
        div_t              R14:2,R12:2    <RETURN>
        uint               R12:2          dividend
        uint               R14:2          divisor
                        div                                        XREF[5]:     FUN_ee16:f0c2(c),
                                                                                FUN_ee16:f12c(c),
                                                                                FUN_fa90:faa8(c),
                                                                                FUN_fb7e:fb9e(c),
                                                                                FUN_ff2a:ff30(c)
        fcea 0f 4e           MOV.W      divisor,R15
        fcec 0d 4c           MOV.W      dividend,R13
        fcee 0e 43           MOV.W      #0,divisor
        fcf0 1c 43           MOV.W      #1,dividend

                        LAB_fcf2                                   XREF[1]:     fcfe(j)
        fcf2 0d 5d           RLA.W      R13
        fcf4 0e 6e           RLC.W      divisor
        fcf6 0e 9f           CMP.W      R15,divisor
        fcf8 01 28           JNC        LAB_fcfc
        fcfa 0e 8f           SUB.W      R15,divisor

                        LAB_fcfc                                   XREF[1]:     fcf8(j)
        fcfc 0c 6c           RLC.W      dividend
        fcfe f9 2b           JNC        LAB_fcf2
        fd00 30 41           RET
```

Decompile: div - (firmware_dw12.hex)

```
1
2 div_t div(uint dividend,uint divisor)
3
4 {
5   bool bVar1;
6   bool bVar2;
7   bool bVar3;
8   uint uVar4;
9   div_t dVar5;
10
11  uVar4 = 0;
12  dVar5.quot = 1;
13  do {
14    bVar1 = CARRY2(dividend,dividend);
15    dividend = dividend * 2;
16    uVar4 = uVar4 * 2 + (uint)bVar1;
17    bVar1 = divisor <= uVar4;
18    if (bVar1) {
19      uVar4 = uVar4 - divisor;
20    }
21    bVar2 = CARRY2(dVar5.quot,(uint)bVar1);
22    bVar3 = CARRY2(dVar5.quot,dVar5.quot + bVar1);
23    dVar5.quot = dVar5.quot * 2 + (uint)bVar1;
24  } while (!bVar2 && !bVar3);
25  dVar5.rem = uVar4;
26  return dVar5;
27 }
28
```

Fixing Function Signatures - can fix a lot of stuff

```
                    LAB_f0ac                                XREF[2]:     f09a(j), f0a0(j)
f0ac e2 b3 c7 02    BIT.B       #2,&DAT_02c7                             = ??
f0b0 12 28          JNC         LAB_f0d6
f0b2 f2 90 07       CMP.B       #0x7,&DAT_02bd                           = ??
     00 bd 02
f0b8 0e 20          JNE         LAB_f0d6
f0ba 1c 42 78 02    MOV.W       &DAT_0278,R12                            = ??
f0be 3e 40 07 00    MOV.W       #0x7,R14
f0c2 b0 12 ea fc    CALL        #FUN_fcea                                undefined FUN_fcea()
f0c6 82 4c 02 02    MOV.W       R12,&DAT_0202                            = ??
f0ca 1c 42 02 02    MOV.W       &DAT_0202,R12                            = ??
f0ce b0 12 ca fc    CALL        #FUN_fcca                                undefined FUN_fcca()
f0d2 f2 42 bd 02    MOV.B       #8,&DAT_02bd                             = ??

                    LAB_f0d6                                XREF[2]:     f0b0(j), f0b8(j)
f0d6 e2 93 11 02    CMP.B       #2,&DAT_0211                             = ??
f0da 48 23          JNE         LAB_ef6c
f0dc d2 92 0e       CMP.B       &DAT_020e,&DAT_0208                      = ??
     02 08 02
f0e2 44 2b          JNC         LAB_ef6c
```

```
134     FUN_fef6();
135     DAT_0206 = 0x17;
136     DAT_028a = 1;
137   }
138   if (((char)DAT_02c7 < '\0') && (DAT_02b5 == '\0')) {
139     DAT_02c7 = DAT_02c7 & 0x7f;
140     FUN_f78c();
141   }
142   if (((DAT_02c7 & 2) != 0) && (DAT_02bd == '\a')) {
143     uVar2 = DAT_0278;
144     FUN_fcea();
145     DAT_0202 = uVar2;
146     FUN_fcca();
147     DAT_02bd = '\b';
148   }
149   if (((DAT_0211 == 2) && (DAT_020e <= DAT_0208)) && (DAT_02b5 == '\0')) {
150     bVar1 = BCSCTL1;
151     BCSCTL1 = bVar1 | 10;
152     bVar1 = BCSCTL1;
153     BCSCTL1 = bVar1 & 0xfa;
154     bVar1 = DCOCTL;
```

```
                    LAB_f0ac                                XREF[2]:     f09a(j), f0a0(j)
f0ac e2 b3 c7 02    BIT.B       #2,&DAT_02c7                             = ??
f0b0 12 28          JNC         LAB_f0d6
f0b2 f2 90 07       CMP.B       #0x7,&DAT_02bd                           = ??
     00 bd 02
f0b8 0e 20          JNE         LAB_f0d6
f0ba 1c 42 78 02    MOV.W       &DAT_0278,R12                            = ??
f0be 3e 40 07 00    MOV.W       #0x7,R14
f0c2 b0 12 ea fc    CALL        #div                                     div_t div(uint dividend, ui
f0c6 82 4c 02 02    MOV.W       R12,&DAT_0202                            = ??
f0ca 1c 42 02 02    MOV.W       &DAT_0202,R12                            = ??
f0ce b0 12 ca fc    CALL        #FUN_fcca                                undefined FUN_fcca()
f0d2 f2 42 bd 02    MOV.B       #8,&DAT_02bd                             = ??

                    LAB_f0d6                                XREF[2]:     f0b0(j), f0b8(j)
f0d6 e2 93 11 02    CMP.B       #2,&DAT_0211                             = ??
f0da 48 23          JNE         LAB_ef6c
f0dc d2 92 0e       CMP.B       &DAT_020e,&DAT_0208                      = ??
     02 08 02
f0e2 44 2b          JNC         LAB_ef6c
```

```
134     FUN_fef6();
135     DAT_0206 = 0x17;
136     DAT_028a = 1;
137   }
138   if (((char)DAT_02c7 < '\0') && (DAT_02b5 == '\0')) {
139     DAT_02c7 = DAT_02c7 & 0x7f;
140     FUN_f78c();
141   }
142   if (((DAT_02c7 & 2) != 0) && (DAT_02bd == '\a')) {
143     dVar3 = div(DAT_0278,7);
144     iVar2 = dVar3.quot;
145     DAT_0202 = iVar2;
146     FUN_fcca();
147     DAT_02bd = '\b';
148   }
149   if (((DAT_0211 == 2) && (DAT_020e <= DAT_0208)) && (DAT_02b5 == '\0')) {
150     bVar1 = BCSCTL1;
151     BCSCTL1 = bVar1 | 10;
152     bVar1 = BCSCTL1;
153     BCSCTL1 = bVar1 & 0xfa;
154     bVar1 = DCOCTL;
```

Inlining Functions

```
eb78 b0 12 c2 ed     CALL       #FUN_edc2                                undefined FUN_edc2()
eb7c 5c 4b a8 02     MOV.B      0x2a8(R11)=>DAT_02b2,R12                 = ??
eb80 b0 12 f4 fb     CALL       #FUN_fbf4                                undefined FUN_fbf4()
eb84 7b 53           ADD.B      #-1,R11
eb86 7b 90 03 00     CMP.B      #0x3,R11
eb8a f5 2f           JC         LAB_eb76
eb8c 5f 42 b3 02     MOV.B      &DAT_02b3,R15                            = ??
eb90 b0 12 b8 ed     CALL       #FUN_edb8                                undefined FUN_edb8()
eb94 0c 4a           MOV.W      R10,R12
eb96 b0 12 dc ed     CALL       #FUN_eddc                                undefined FUN_eddc()
eb9a b0 12 f4 fb     CALL       #FUN_fbf4                                undefined FUN_fbf4()

                LAB_eb9e                                                 XREF[2]:     eac2(j), eb0a(j)
eb9e 36 3f           JMP        LAB_ea0c
```

```
52    }
53    puVar3 = &DAT_02ab;
54    FUN_fb22(&DAT_02ab,9);
55    uVar2 = 10;
56    do {
57      FUN_edc2();
58      FUN_fbf4(*(undefined1 *)(uVar2 + 0x2a8));
59      bVar1 = (char)uVar2 - 1;
60      uVar2 = (uint)bVar1;
61    } while (2 < bVar1);
62    FUN_edb8();
63    FUN_eddc(puVar3);
64    FUN_fbf4();
65  }
66  return;
```

```
                ***************************************************************
                *                          FUNCTION                           *
                ***************************************************************
                  undefined FUN_edb8()
          undefined       <UNASSIGNED>   <RETURN>
                FUN_edb8                                                 XREF[3]:     FUN_ea12:eb90(c),
                                                                                      FUN_ebe0:ed6e(c),
                                                                                      FUN_ebe0:ed92(c)
          edb8 3e 40 a9 02     MOV.W      #0x2a9,R14
          edbc 3d 40 a8 02     MOV.W      #0x2a8,R13
          edc0 30 41           RET
```

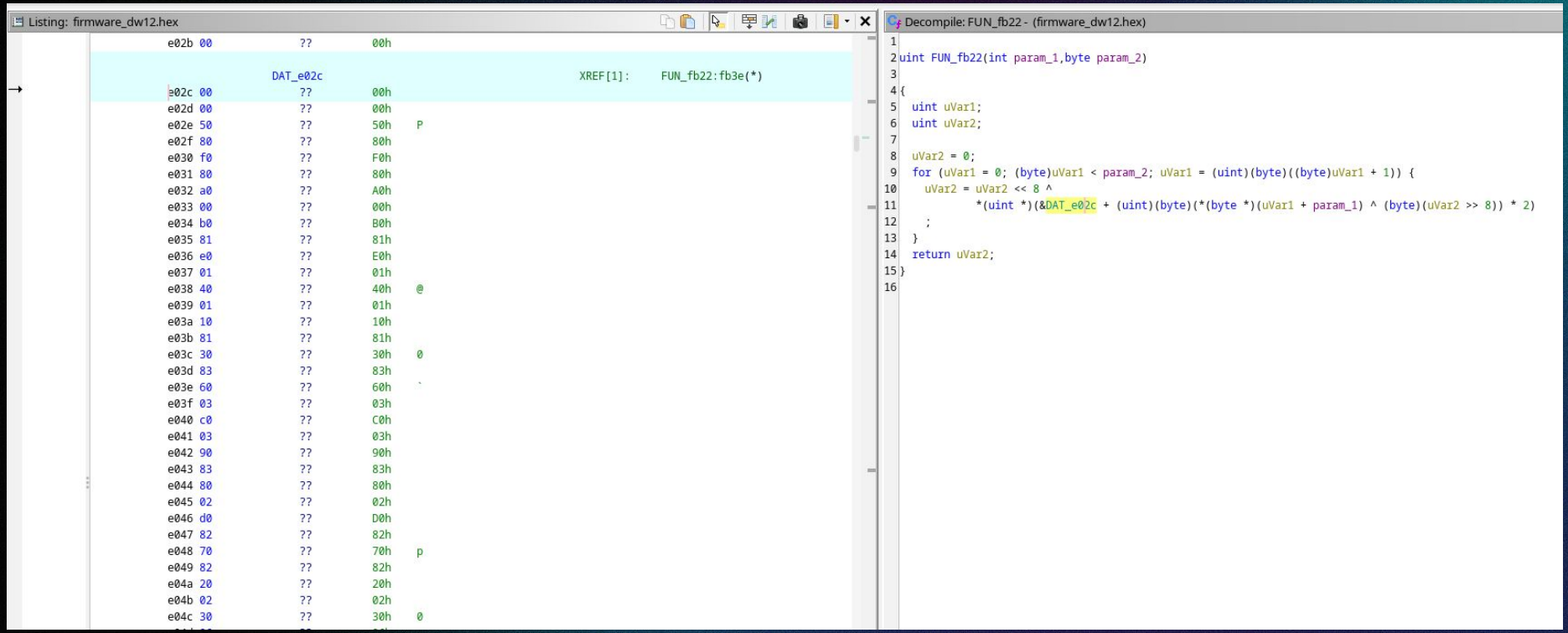
```
1
2  void FUN_edb8(void)
3
4  {
5    return;
6  }
7
```

```
eb78 b0 12 c2 ed     CALL       #FUN_edc2                                undefined FUN_edc2()
eb7c 5c 4b a8 02     MOV.B      0x2a8(R11)=>DAT_02b2,R12                 = ??
eb80 b0 12 f4 fb     CALL       #FUN_fbf4                                undefined FUN_fbf4()
eb84 7b 53           ADD.B      #-1,R11
eb86 7b 90 03 00     CMP.B      #0x3,R11
eb8a f5 2f           JC         LAB_eb76
eb8c 5f 42 b3 02     MOV.B      &DAT_02b3,R15                            = ??
eb90 b0 12 b8 ed     CALL       #FUN_edb8                                undefined FUN_edb8()
eb94 0c 4a           MOV.W      R10,R12
eb96 b0 12 dc ed     CALL       #FUN_eddc                                undefined FUN_eddc()
eb9a b0 12 f4 fb     CALL       #FUN_fbf4                                undefined FUN_fbf4()

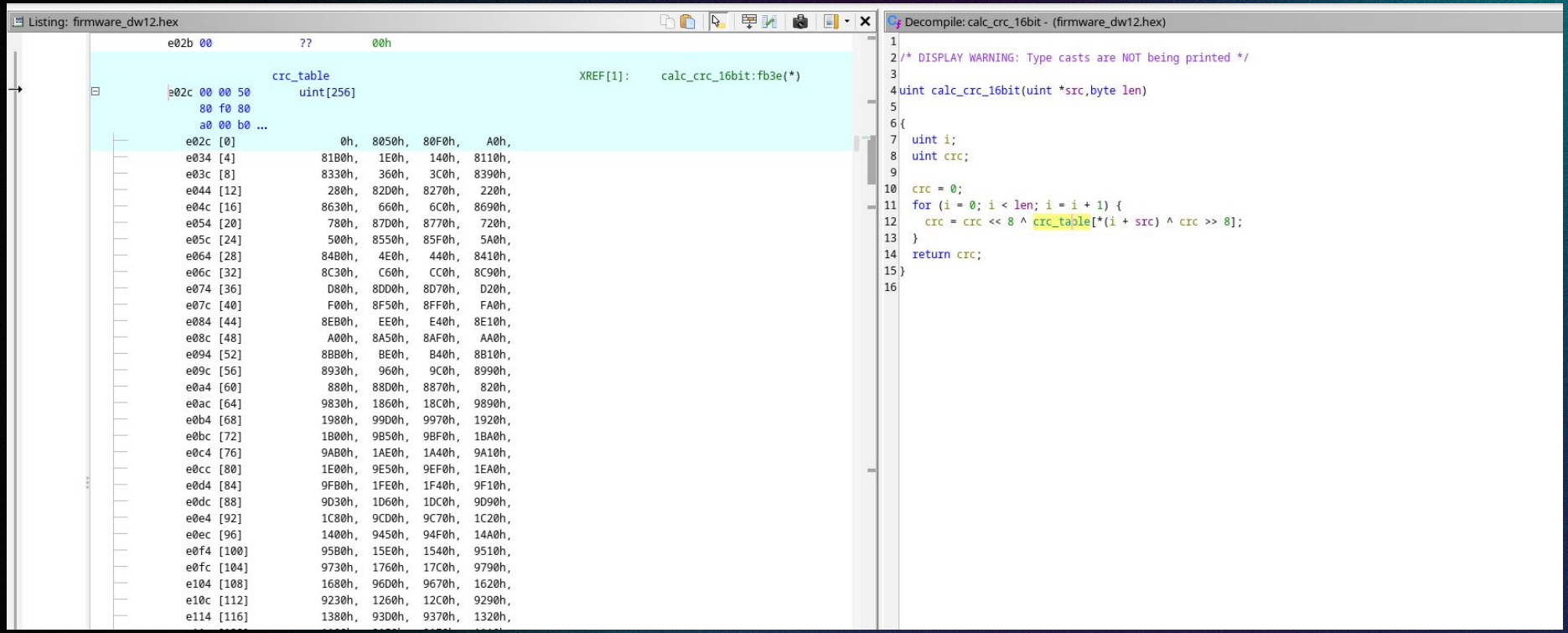
                LAB_eb9e                                                 XREF[2]:     eac2(j), eb0a(j)
eb9e 36 3f           JMP        LAB_ea0c
```

```
52    FUN_eba0();
53    DAT_02b3 = (DAT_02c5 ^ 0x10) & 0xf0;
54  }
55  puVar3 = &DAT_02ab;
56  FUN_fb22(&DAT_02ab,9);
57  uVar2 = 10;
58  do {
59    FUN_edc2();
60    FUN_fbf4(*(undefined1 *)(uVar2 + 0x2a8));
61    bVar1 = (char)uVar2 - 1;
62    uVar2 = (uint)bVar1;
63  } while (2 < bVar1);
64  FUN_eddc(puVar3,0x2a8,&DAT_02a9,DAT_02b3);
65  FUN_fbf4();
66  }
```

Define Types and Arrays



Define Types and Arrays



We can see the Manchester encoding LUT

```
e02c 00 00 50        uint[256]
     80 f0 80
     a0 00 b0 ...

                     machester_encoding[10]                XREF[1,1]:   FUN_e982:e9f0(R),
                     machester_encoding                                 FUN_e982:e9a4(R)
e22c aa a9 a6        db[16]
     a5 9a 99
     96 95 6a ...
   e22c [0]            AAh, A9h, A6h, A5h,
   e230 [4]            9Ah, 99h, 96h, 95h,
   e234 [8]            6Ah, 69h, 66h, 65h,
   e238 [12]           5Ah, 59h, 56h, 55h
e23c ff              ??         FFh
e23d ff              ??         FFh
e23e 40              ??         40h    @
e23f 38              ??         38h    8
e240 17              ??         17h
e241 00              ??         00h
e242 20              ??         20h
e243 ff              ??         FFh
e244 ff              ??         FFh
e245 01              ??         01h
e246 fd              ??         FDh
e247 1d              ??         1Dh
e248 20              ??         20h
e249 0a              ??         0Ah
e24a 0a              ??         0Ah
e24b 0a              ??         0Ah
e24c 01              ??         01h
e24d ff              ??         FFh
e24e ff              ??         FFh
e24f ff              ??         FFh
e250 ff              ??         FFh
e251 ff              ??         FFh
```

```
1
2 void FUN_fbf4(byte param_1,byte *param_2,byte *param_3)
3
4 {
5   byte bVar1;
6
7   bVar1 = param_1;
8   FUN_fd28();
9   *param_2 = machester_encoding[bVar1];
10  *param_3 = machester_encoding[param_1 & 0xf];
11  return;
12 }
13
```

Starting to see the picture.
So what is our device doing?

```
38      if (DAT_0211 != '\x02') {
39        return;
40      }
41      buffer2[2] = 0x7a;
42      buffer2[3] = packet_counter >> 8;
43      buffer2[4] = packet_counter;
44      buffer2[5] = (DAT_0228 & 1) * '\x04' | event_data & 0xf9;
45      buffer2[5] = buffer2[5] ^ some_secret;
46      buffer2[5] = buffer2[5] & 0xfc;
47      special_high_byte = (another_secret_maybe ^ 0x10) & 0xf0;
48    }
49    buffer2[9] = DAT_0216;
50    buffer2[8] = DAT_0215;
51    buffer2[7] = DAT_0214;
52    buffer2[6] = DAT_0213;
53    DAT_020e = 0x18;
54    uVar2 = calc_crc_16bit(buffer2 + 2,9);
55    len = 10;
56    do {
57      get_manchester_encoding(*(len + 0x2a8),&DAT_0292 + len * 2,&event_data + len * 2);
58      bVar1 = len - 1;
59      len = bVar1;
60    } while (2 < bVar1);
61    get_manchester_encoding(special_high_byte | uVar2 >> 0xc,0x2a8,buffer2);
62    get_manchester_encoding(uVar2 >> 4,buffer2 + 1,buffer2 + 2);
63    return;
64  }
```

```
PRE:16h SYNC?8h COUNTER:16d DATA:8b ID 12d 20d CHK?16h

Link to this data and format (Click c
[TXID: 0056-040-5817]
[closed]
PRE:fffe SYNC?7a COUNTER:00467 DATA:01100100 ID 0056 0405817 CHK?3a4b
PRE:fffe SYNC?7a COUNTER:00469 DATA:00011000 ID 0056 0405817 CHK?2a74
PRE:fffe SYNC?7a COUNTER:00471 DATA:00011000 ID 0056 0405817 CHK?2eb7
PRE:fffe SYNC?7a COUNTER:00473 DATA:11110000 ID 0056 0405817 CHK?c0ef
PRE:fffe SYNC?7a COUNTER:00475 DATA:11001000 ID 0056 0405817 CHK?e475
```

Oh, look! Rabbit food!

```
                    //
                    // CODE
                    // Generated by Intel Hex
                    // RAM:e000-RAM:ffff
                    //

                    some_lut[1]                                     XREF[0,3]:   FUN_f36a:f3e8(R),
                    some_lut[1]+2                                                FUN_f36a:f3e8(R),
                    some_lut[2]                                                  FUN_f36a:f3ec(R)
                    some_lut
       e000 4d d3 34        ulong[8]
            4d d3 34
            4d d3 34 ...
         e000 [0]                4D34D34Dh,    D34D34D3h,    34D34D34h,    4D34D34Dh,
         e010 [4]                D34D34D3h,    34D34D34h,    4D34D34Dh,    D34D34D3h
```

Continue the process until we figure out these things:

- Rabbit uses a 128 bit key, that in our case is generated from a 16 bit seed value
- The seed value is stored in flash in the INFO section
- 4 bits of the raw cipher are being sent with 12 bits of the CRC as an additional MIC check
- The 128 bit key is regenerated every 12 packets, based on the packet counter sent with the packet - completely deterministic
- No initialization vector (IV) is used
- Generates the cipher stream on the fly instead of in blocks due to RAM limitations

Cryptographic functions rely on high entropy

Changes in a single bit of the input cause many of bits to change in the output.

If we collect more than 16 bits of the raw cipher, the probability of having a unique sequence of cipher for that input becomes almost 1 for a good cryptographic function.

16 bits of the raw cipher are transmitted every 4 packets, and around every 5 hours.

What next?

Since the cipher works fast on limited hardware, it works even faster on more powerful hardware.

AI was able to write an emulator for the firmware that worked to decode some values, but we can do better. We now know the entire system.

16 bit space is trivial to search, and we can use the MIC to check if the 16 bit seed is the correct one.

Whittle possible seeds down until only one seed produces a matching cipher stream from the captured packets.

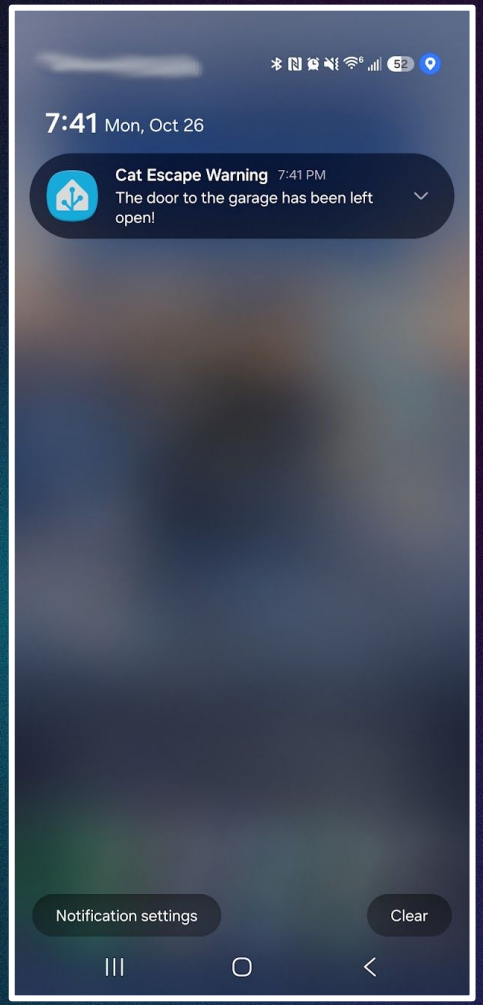
Pull request back to rtl_433!

Bonus!

One of the other people interested in using these devices wants to decode these packets on an ESP32 with a CC1101 chip, similar to a Flipper Zero.

The code runs well and can determine the seed even on an ESP32 running at 240MHz after 6 packet captures in a matter of seconds.

Profit!


7:41 Mon, Oct 26
Cat Escape Warning 7:41 PM
The door to the garage has been left open!
Notification settings
Clear

Noteable Additional Resources

HackadayU: Reverse Engineering with Ghidra - In depth Ghidra class

DEF CON 32: Anyone can hack IoT

Many more! Go forth and reverse engineer!